

Módulo de Atendimento por Voz

VoIP · SIP Trunk · Agentes de IA · Painei 360

Análise de necessidade, viabilidade técnica e arquitetura proposta

Plataforma RAPIA · Infoprime

Documento técnico de análise · versão 1.0 · 19/09/2026

Elaborado com apoio do Claude (Anthropic) a partir do descritivo do módulo e do código atual
(sysapi: Laravel 8 / PHP 7.4 · sysweb: Vue 2.5 / Webpack 3).

Sumário

Sumário	2
1. Sumário executivo	3
2. Entendimento da necessidade	3
2.1 Cenários de negócio descritos	3
2.2 Requisitos não funcionais implícitos	4
2.3 O que já existe no RAPIA e pode ser reaproveitado	4
3. O LiveKit é a melhor alternativa?	5
3.1 O que o LiveKit oferece	5
3.2 Comparação com as alternativas	5
3.3 Pontos de atenção com operadoras brasileiras	5
3.4 Fase 2: chamadas de voz pela API oficial do WhatsApp	6
3.5 Veredito	6
4. Modelo de IA: Gemini Live x OpenAI Realtime x pipeline	6
5. Compatibilidade com o stack atual (Vue 2.5 + Laravel 8)	7
5.1 Backend (sysapi – Laravel 8 / PHP 7.4)	7
5.2 Frontend (sysweb – Vue 2.5 / Webpack 3 / Babel 6)	8
5.3 Painel 360: rota web do atendente	8
5.4 Débito técnico: atenção, sem bloqueio	8
6. Arquitetura proposta	8
6.1 Visão de componentes	8
6.2 Fluxos por cenário	9
6.3 Modelo de dados sugerido (migrations com a checagem de banco "rapia")	10
7. Ambiente de desenvolvimento local (docker-compose + ngrok)	10
7.1 Resposta direta	10
7.2 Três formas viáveis de desenvolver	10
7.3 Esqueleto de docker-compose (opção A/B)	11
8. Configuração dos agentes de IA dentro do RAPIA	11
8.1 Telas sugeridas	11
8.2 Cuidados de segurança e governança	12
9. Produção na AWS: mesmo servidor ou instância separada?	12
9.1 Resposta direta	12
9.2 Topologia recomendada	12
9.3 Por que o LiveKit não deve entrar no Swarm (e os workers sim)	13
9.4 Portas e segurança (Security Group)	13
9.5 Self-hosted x LiveKit Cloud	13
10. Regras de negócio e pontos de atenção	13
11. Riscos e mitigação	14
12. Roadmap sugerido	14
13. Conclusão e próximos passos	15

1. Sumário executivo

Este documento analisa a proposta de um **módulo de atendimento por voz** no RAPIA, cobrindo atendimento autônomo por agente de IA, triagem com transbordo para atendente humano, continuidade via WhatsApp e fila de retorno de chamadas (callback). A análise considera o código atual do projeto (*sysapi* em Laravel 8 / PHP 7.4, *sysweb* em Vue 2.5 / Webpack 3, Redis como fila, Pusher para eventos em tempo real, MySQL, Flowise para copiloto de IA) e o estado das tecnologias em setembro de 2026.

Respostas curtas às cinco perguntas levantadas:

Pergunta	Resposta	Condição / observação
O LiveKit é a melhor alternativa?	Sim, é a opção recomendada para o perfil do RAPIA (voz com IA + atendente via navegador + fase 2 WhatsApp).	O LiveKit SIP não faz SIP REGISTER . É preciso confirmar que a operadora entrega o tronco por IP (autenticação por IP ou digest no sentido de saída). Se o tronco só funcionar por registro, entra um SBC (Asterisk/Kamailio) na frente. A voz do WhatsApp via <i>LiveKit Connectors</i> hoje é exclusiva do LiveKit Cloud .
Vue 2.5 + Laravel 8 comportam?	Sim . O Laravel gera tokens, orquestra filas e callbacks e recebe webhooks. O Vue 2 roda o <i>livekit-client</i> (WebRTC) no Paine 360.	O agente de voz não roda em PHP : é preciso criar um novo serviço (worker Python/Node com LiveKit Agents). O Webpack 3/Babel 6 provavelmente não transpila o SDK atual, então carregue o bundle UMD. Recomenda-se planejar a atualização do stack, que já está sem suporte (EOL).
LiveKit local via docker-compose + ngrok para o tronco?	Parcialmente . O docker-compose local funciona. O ngrok não serve para o tronco , porque não tunela UDP e o áudio (RTP) é UDP.	Recomendado: LiveKit + SIP em uma EC2 de desenvolvimento pequena (IP público) ou no LiveKit Cloud, com o worker de IA e o Laravel rodando localmente. O ngrok continua útil para receber webhooks HTTP no Laravel local.
Configurar agentes de IA dentro do RAPIA?	Sim, e é o desenho recomendado .	O RAPIA guarda prompts versionados, <i>tools</i> (JSON Schema + endpoint), voz/modelo e troncos. O worker carrega essa configuração no início de cada chamada. O RAPIA sincroniza troncos e regras com a API do LiveKit.
O LiveKit precisa estar no mesmo servidor do RAPIA?	Não, e não deve estar .	Use uma EC2 dedicada com IP elástico e rede <i>host</i> (faixas UDP amplas). Isso não combina com a malha de ingress do Swarm. Os workers de IA, por outro lado, podem rodar no Swarm.

Recomendação geral: prosseguir com LiveKit (self-hosted na AWS, região sa-east-1) + LiveKit Agents (Python) + Gemini Live como modelo padrão, com OpenAI Realtime como alternativa configurável. Começar por uma **prova de conceito de 2 a 3 semanas** que valide três riscos antes de tudo: (1) compatibilidade do tronco da operadora (IP x REGISTER, codec, caller ID), (2) qualidade da IA em português para capturar CPF, datas e nomes por telefone e (3) o *livekit-client* no build atual do *sysweb*.

2. Entendimento da necessidade

2.1 Cenários de negócio descritos

#	Cenário	Fluxo resumido	Capacidades exigidas
1	Autoagendamento por voz	Paciente liga → IA pede CPF → consulta/cadastra paciente → conduz agendamento (especialidade, profissional, convênio, data/hora) → confirma → encerra.	SIP de entrada, IA speech-to-speech, <i>function calling</i> para as APIs da clínica, captura robusta de CPF/datas.

#	Cenário	Fluxo resumido	Capacidades exigidas
2	Triagem + transbordo humano	IA faz a triagem → grava dados estruturados no RAPIA → coloca na fila de voz → atendente vê no Painel 360 → assume a ligação pelo navegador.	Fila de voz em tempo real, espera (música/mensagens), handoff IA→humano sem perder a ligação, WebRTC no navegador.
3	Sem atendente → WhatsApp	Sem atendente disponível, a IA oferece seguir pelo WhatsApp → paciente aceita → template disparado → resposta abre a janela de 24h → chat convencional (módulo existente).	Disponibilidade dos atendentes, confirmação do número, disparo de template, vínculo chamada ↔ sessão de chat.
4	Callback (retorno)	Paciente recusa o WhatsApp → IA avisa que um atendente vai retornar → entra na fila de saída → primeiro atendente online recebe um modal → confirma → sistema origina a ligação.	Fila de callback persistente, distribuição atômica, originação SIP (saída), regras de tentativa e horário.

2.2 Requisitos não funcionais implícitos

- **Latência conversacional:** resposta da IA percebida abaixo de ~1 s. Isso favorece modelos speech-to-speech, servidor de mídia no Brasil (sa-east-1) e ferramentas (tools) rápidas, idealmente abaixo de 1,5 s.
- **Continuidade da chamada:** a passagem IA → fila → humano não pode derrubar a ligação nem exigir nova discagem.
- **Dados de saúde (LGPD, art. 11):** triagem e agendamento envolvem dado pessoal sensível. É preciso aviso de gravação, base legal, contrato de tratamento com o provedor de IA (transferência internacional), retenção e controle de acesso às gravações e transcrições.
- **Segurança clínica:** a IA de triagem não pode dar orientação médica e deve reconhecer sinais de emergência, orientando o paciente a ligar 192/193 ou encaminhando com prioridade.
- **Multi-tenant:** o RAPIA usa subdomínios por instituição. Troncos, números, agentes, filas e gravações precisam ser isolados por tenant.
- **Observabilidade e indicadores:** TME, TMA, abandono, taxa de resolução pela IA, custo por minuto e transcrições auditáveis, integrados aos relatórios existentes.

2.3 O que já existe no RAPIA e pode ser reaproveitado

Ativo existente	Reaproveitamento no módulo de voz
Filas (Queues, UsersHasQueues, QueuesHasDay, QueueHasLimits)	Base para as filas de voz: vínculo de atendentes, horários e limites. Recomenda-se um tipo/canal "voz" na fila, em vez de uma estrutura paralela.
Mapa de agentes (MapaAgentesController) e status online/pausa	Fonte da disponibilidade para os cenários 3 e 4 e base visual do Painel 360.
Pusher/Echo + canais privados, EventBatcher, services de cache em Redis	Eventos em tempo real do Painel 360: chamada entrou na fila, IA triando, chamada assumida, callback disponível.
WhatsTemplateController + API Cloud da Meta + sessões de chat	Cenário 3: disparo do template e abertura da sessão de 24h no chat já existente.
Contatos (ContactsController), sessões, notas, tags	A chamada vira uma sessão do contato, com histórico omnichannel (voz + WhatsApp).
Integração Flowise (copiloto/agente virtual)	Não serve para áudio em tempo real, mas as mesmas APIs de negócio podem ser expostas como tools do agente de voz.
Filas Redis (QUEUE_CONNECTION=redis) e S3	Jobs de pós-chamada (resumo, transcrição, métricas) e armazenamento das gravações (Egress → S3).

3. O LiveKit é a melhor alternativa?

3.1 O que o LiveKit oferece

- **LiveKit Server:** SFU WebRTC open source (Apache 2.0, escrito em Go). Cada ligação vira uma *sala* onde entram o paciente (via SIP), o agente de IA e, depois, o atendente humano pelo navegador.
- **LiveKit SIP:** ponte SIP ↔ sala. Tem troncos de entrada e de saída, *dispatch rules* (qual número vai para qual sala/agente), DTMF (RFC 4733), transferência *cold* (SIP REFER) e *warm*, e originação de chamadas (CreateSIPParticipant).
- **LiveKit Agents:** framework (Python e Node.js) para agentes de voz, com plugins nativos para Gemini Live, OpenAI Realtime e pipelines STT → LLM → TTS. Inclui detecção de turno, interrupção (barge-in), *function tools* e despacho de jobs com metadados.
- **Egress:** gravação de salas/participantes direto para S3.
- **Connectors (set/2026):** ponte nativa para chamadas de voz do WhatsApp Business (API *AcceptWhatsAppCall* a partir do SDP enviado pela Meta no webhook). **Hoje disponível apenas no LiveKit Cloud.**

O ganho principal para o RAPIA é que a ligação telefônica, a IA e o atendente no navegador ficam **na mesma sala**. O handoff do cenário 2 deixa de ser uma transferência de telefonia (REFER, ramal, PBX) e passa a ser uma operação de aplicação: o atendente entra na sala e o agente de IA sai. Isso é mais simples, confiável e instantâneo do que com um PBX tradicional.

3.2 Comparação com as alternativas

Alternativa	Pontos fortes	Pontos fracos para o RAPIA	Adequação
LiveKit (self-hosted ou Cloud)	SIP + WebRTC + agentes de IA integrados; SDK de navegador maduro; open source; comunidade grande em voice AI; caminho para o WhatsApp (Connectors).	Sem SIP REGISTER; exige infraestrutura com UDP aberto; worker em Python/Node (novo stack); WhatsApp nativo só no Cloud.	Alta (recomendado)
Asterisk / FreeSWITCH + IA (AudioSocket, mod_audio_stream, Pipecat)	Telefonia madura; suporte a REGISTER; URA, filas e gravação consolidadas; muita mão de obra no Brasil.	Atendente no navegador exige WebRTC/SIP.js + TURN; integração com IA feita "na mão"; handoff por transferência de canal; mais código de baixo nível.	Média. Útil como SBC na frente do LiveKit, se necessário.
Jambonz (CPaaS open source)	Feito para voice AI sobre SIP; suporta registro em operadoras; webhooks de controle de chamada.	Comunidade menor; atendente web depende de outra peça; menos pronto para o cenário "sala com humano + IA".	Média
CPaaS (Twilio, Vonage, Plivo)	Nada de infraestrutura de mídia; Media Streams para IA; SDK de voz no navegador.	Custo por minuto em dólar e números/tronco da própria CPaaS (não da operadora atual); dependência de fornecedor.	Média/baixa: bom para PoC, caro em escala.
SaaS de voice AI (Vapi, Retell, Bland etc.)	Muito rápido para colocar a IA no ar.	Pouco controle sobre dados de saúde e sobre o handoff para o painel humano; custo alto; difícil embutir no RAPIA como produto.	Baixa para o produto final

3.3 Pontos de atenção com operadoras brasileiras

1. Registro SIP (REGISTER) não é suportado pelo LiveKit SIP. A documentação oficial marca "SIP Registration (REGISTER): Not Supported". Muitas operadoras e revendas VoIP no Brasil entregam o tronco no modelo *registro com usuário e senha*. O LiveKit espera que a operadora envie os INVITES para o IP dele (autenticação por IP / IP

liberado) e, na saída, autentica por digest a cada chamada.

- **Pergunte à operadora primeiro:** ela entrega o tronco por IP fixo (peer/IP auth)? Se sim, o LiveKit conecta direto.
- **Se só houver registro:** coloque um SBC leve (Asterisk com PJSIP ou Kamailio) na mesma VPC. Ele registra na operadora e encaminha as chamadas por IP ao LiveKit SIP. Custo baixo, e ainda ganha controle de normalização de números, codecs e failover entre operadoras.
- **Codec:** no Brasil o padrão é G.711 A-law (PCMA), que o LiveKit SIP suporta. Valide também G.729, caso a operadora o imponha (se for o caso, o SBC transcodifica).
- **Caller ID / BINA:** confirme que o número de origem chega no cabeçalho (From/P-Asserted-Identity) e que a operadora permite apresentar o número da central nas chamadas de retorno (cenário 4).
- **DTMF:** confirme o envio por RFC 2833/4733 (não SIP INFO). Isso abre a opção de o paciente **digitar o CPF no teclado**, bem mais confiável que o reconhecimento de fala para 11 dígitos.
- **Canais simultâneos e numeração:** confira a quantidade de canais contratados, a portabilidade dos DIDs atuais da central e as regras de discagem (DDD, 0XX, celular).

3.4 Fase 2: chamadas de voz pela API oficial do WhatsApp

A **WhatsApp Business Calling API** (aberta em julho de 2025) fala WebRTC nativamente e também permite encaminhar chamadas para infraestrutura SIP. Hoje há três caminhos com LiveKit:

Caminho	Como funciona	Situação em set/2026
LiveKit Connectors (Cloud)	O webhook de chamada da Meta chega no RAPIA, o RAPIA repassa o SDP para AcceptWhatsAppCall e o chamador entra na sala como qualquer participante.	GA no LiveKit Cloud (cobrança adicional por minuto). Não anunciado para o self-hosted.
WhatsApp → SIP → LiveKit SIP (self-hosted)	A Meta encaminha a chamada via SIP para o seu servidor.	Exige Opus e DTLS-SRTP/ICE-lite no lado SIP. Há contribuição em andamento no livekit/sip (PR #842). Considerar experimental.
WhatsApp → SBC (FreeSWITCH/Asterisk) → LiveKit SIP	Um SBC termina a mídia da Meta e entrega G.711/Opus ao LiveKit.	Viável, com mais uma peça para operar.

Implicação arquitetural: o LiveKit é uma boa aposta para a fase 2, porque o WhatsApp vira só mais uma "porta de entrada" para a mesma sala, com os mesmos agentes, fila e painel. Na hora da fase 2, porém, será preciso decidir entre (a) migrar ou usar em modo híbrido o LiveKit Cloud, (b) esperar o suporte self-hosted amadurecer ou (c) operar um SBC. Por isso o código do RAPIA e do worker deve tratar a **origem da chamada como um atributo** (pstn | whatsapp | web), nunca como um fluxo separado.

3.5 Veredito

O LiveKit é, hoje, a melhor relação entre capacidade e esforço para os quatro cenários. A escolha vem com duas condições objetivas: validar o modelo de tronco da operadora (IP x REGISTER) e aceitar um novo serviço em Python no ecossistema. Os demais riscos têm mitigação conhecida.

4. Modelo de IA: Gemini Live x OpenAI Realtime x pipeline

Critério	Gemini Live (native audio)	OpenAI Realtime (gpt-realtime)	Pipeline STT → LLM → TTS
Latência	Baixa (speech-to-speech)	Baixa (speech-to-speech)	Média (soma das etapas; ~1–2 s bem otimizado)

Critério	Gemini Live (native audio)	OpenAI Realtime (gpt-realtime)	Pipeline STT → LLM → TTS
Português (BR)	Bom; validar sotaques e ruído de linha 8 kHz	Bom; validar	Excelente com STT/TTS escolhidos para pt-BR
Function calling	Suportado	Suportado e maduro	Máximo controle (qualquer LLM, inclusive Claude)
Custo relativo	Menor entre os realtime	Maior (várias vezes o Gemini)	Variável; em geral competitivo
Controle de voz	Vozes do provedor	Vozes do provedor	Qualquer TTS (ex.: vozes pt-BR dedicadas)
Transcrição/auditoria	Transcrição disponível	Transcrição disponível	Nativa (o STT já é texto)

Recomendação: use o LiveKit Agents como camada de abstração e torne o **provedor/modelo um campo de configuração do agente** no RAPIA. Comece com Gemini Live pelo custo e compare com OpenAI Realtime na PoC, com as mesmas gravações de teste. Mantenha o pipeline cascata como opção para agentes que precisem de uma voz específica ou de um LLM diferente. Os preços por token de áudio mudam com frequência. Antes do business case, meça o custo real por minuto na PoC (a telemetria do LiveKit Agents reporta o uso de tokens).

Boas práticas para telefonia: use DTMF como alternativa para o CPF; peça sempre a confirmação ("confirmando: 123.456.789-00, correto?"); valide o dígito verificador do CPF na tool antes de consultar o ERP; mantenha tools idempotentes (não duplicar agendamentos) e com timeout curto e mensagem de espera falada ("um momento, estou consultando a agenda"); limite o escopo do prompt e registre toda chamada de tool para auditoria.

5. Compatibilidade com o stack atual (Vue 2.5 + Laravel 8)

5.1 Backend (sysapi – Laravel 8 / PHP 7.4)

O Laravel **comporta integralmente** o papel de orquestrador. Nada do que o RAPIA precisa fazer com o LiveKit exige recursos que o PHP 7.4 não tenha:

- **Tokens de acesso:** o token do LiveKit é um JWT HS256 com *grants* (sala, publicar/assinar). Pode ser gerado com *firebase/php-jwt* ou pelo SDK PHP comunitário (verifique a compatibilidade de versão com PHP 7.4).
- **API de servidor:** criar/remover salas, remover participantes, atualizar metadados, criar troncos e dispatch rules, originar chamadas (CreateSIPParticipant) e iniciar gravação (Egress). Tudo é HTTP/JSON (Twirp) autenticado por JWT, e o Guzzle já existente resolve.
- **Webhooks do LiveKit** (room_started, participant_joined/left, egress_ended...): rota POST validando a assinatura JWT do cabeçalho Authorization. Os eventos alimentam o estado da chamada e disparam broadcasts no Pusher.
- **API para o agente de IA:** endpoints autenticados (Passport client credentials ou token de serviço) que o worker consome: carregar configuração do agente, executar tools, registrar triagem, enfileirar, consultar disponibilidade, disparar template do WhatsApp e criar callback.
- **Filas e callbacks:** Redis + jobs já existentes. Para a distribuição atômica do callback (cenário 4), use *lock* do Redis (Cache::lock) ou *SELECT ... FOR UPDATE*, para que dois atendentes nunca peguem o mesmo retorno.

Único componente que não cabe no PHP: o **Voice Agent Worker**. Ele mantém conexões WebRTC e streams de áudio bidirecionais de longa duração com o modelo de IA, então deve ser um serviço separado em **Python** (SDK mais maduro do LiveKit Agents) ou Node.js. O worker é "burro" em regras de negócio: recebe a configuração do RAPIA, executa as tools chamando a API do RAPIA e devolve os resultados. Assim, a lógica de negócio continua centralizada no Laravel.

5.2 Frontend (sysweb – Vue 2.5 / Webpack 3 / Babel 6)

- **livekit-client** (SDK JS do navegador) é agnóstico de framework e funciona em Vue 2: um componente/serviço Vuex encapsula conectar à sala, publicar microfone, anexar o áudio remoto, mute/hold e desligar.
- **Risco de build:** o Webpack 3 e o Babel 6 não interpretam sintaxe moderna (ES2020+, como *optional chaining*), que versões recentes do SDK podem conter. Mitigação: carregar o **bundle UMD** pré-compilado (*livekit-client.umd.js*) como script estático/external, sem passar pelo babel-loader. Valide isso na PoC.
- **Requisitos de navegador:** HTTPS obrigatório (getUserMedia), permissão de microfone, política de autoplay (iniciar o áudio após interação do usuário, que o botão "Assumir" já garante), headset recomendado.
- **Eventos do painel:** via Pusher/Echo já existente (canal privado por tenant/fila). O LiveKit cuida só do áudio. O estado de negócio (fila, triagem, callbacks) vem do RAPIA.

5.3 Painel 360: rota web do atendente

Sim, é possível montar a rota. Composição sugerida:

Área do painel	Conteúdo	Origem dos dados
Barra do atendente	Status (disponível, em chamada, pausa), fila(s) atendidas, microfone/headset, softphone (mute, hold, DTMF, desligar, transferir)	RAPIA + livekit-client
Fila de voz ao vivo	Chamadas aguardando: tempo de espera, prioridade, motivo triado, origem (PSTN/WhatsApp), botão Assumir	Pusher (eventos do RAPIA)
Chamadas com IA em curso	Ligações ainda com o agente de IA, com transcrição parcial ao vivo (opcional) e opção de intervir/assumir	Webhooks LiveKit + eventos do worker
Ficha 360 do paciente	Dados da triagem (JSON estruturado), cadastro, agendamentos, histórico omnichannel (chats WhatsApp, chamadas anteriores, notas, tags)	RAPIA + ERP da clínica
Callbacks	Fila de retornos pendentes, tentativas, modal de oferta ao atendente que fica online	RAPIA (jobs + Pusher)
Supervisão	Chamadas em curso por atendente, escuta silenciosa (entrar na sala só como assinante)	LiveKit (grants somente-assinar)

Origem de chamadas (cenário 4) no navegador: o atendente confirma no modal → o RAPIA cria a sala e entrega o token → o atendente entra primeiro (ouve o "chamando") → o RAPIA chama CreateSIPParticipant no tronco de saída com o número do paciente → o paciente atende e entra na mesma sala. Com isso o paciente nunca atende uma ligação muda.

5.4 Débito técnico: atenção, sem bloqueio

PHP 7.4, Laravel 8 e Vue 2 estão fora de suporte (EOL). Isso **não impede** o módulo, porque a integração com o LiveKit é via HTTP/JWT e bundle UMD. Mas um módulo novo, com dados de saúde e exposição a telefonia, aumenta a superfície de risco. Recomenda-se: (a) escrever o módulo de voz de forma isolada (namespace/rotas próprios, componentes Vue desacoplados) para facilitar a migração futura e (b) incluir no roadmap a subida para PHP 8.2+ / Laravel 10+ e a estratégia para o frontend (Vue 2.7 como degrau intermediário, ou Vue 3 + Vite).

6. Arquitetura proposta

6.1 Visão de componentes

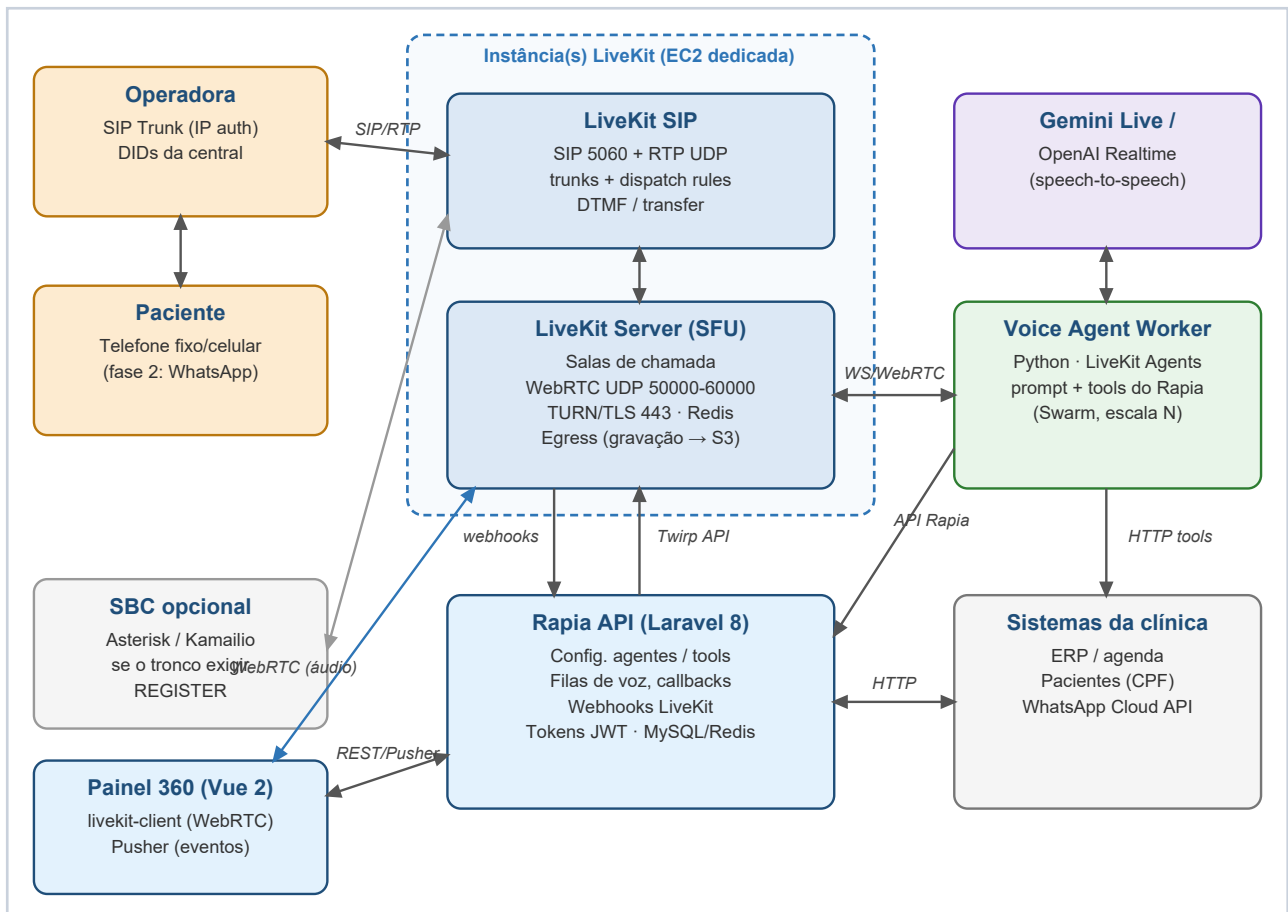


Figura 1: componentes do módulo de voz. O SBC só é necessário se o tronco exigir REGISTER. O Voice Agent Worker conecta-se de dentro para fora ao LiveKit (não precisa de porta aberta), por isso pode rodar no Swarm ou até na máquina do desenvolvedor.

6.2 Fluxos por cenário

Cenário 1: Autoagendamento

```

Operadora --INVITE--> LiveKit SIP --dispatch rule (DID -> tenant/agente)--> sala "call-{tenant}-{uuid}"
LiveKit --job--> Voice Agent Worker (metadados: tenant, agent_id, caller_number, did)
Worker --GET /voice/agents/{id}/runtime--> RAPIA (prompt, tools, voz, modelo, regras)
IA: saudação + aviso LGPD/gravação -> pede CPF (fala ou DTMF) -> confirma
    tool buscar_paciente(cpf) -> RAPIA -> ERP (não achou -> tool cadastrar_paciente)
    tool listar_especialidades / convenios
    tool buscar_horarios(especialidade, profissional?, periodo)
    tool agendar(paciente, horario) -> idempotency-key = call_id + slot
IA confirma data/hora/local/preparo -> tool finalizar(resultado=agendado) -> encerra a ligação
RAPIA: grava a sessão de voz, transcrição, resumo, tools executadas; (opcional) envia confirmação por WhatsApp
    
```

Cenário 2: Triagem + atendente humano

```

IA coleta os dados definidos no "formulário de triagem" do agente (JSON Schema configurável)
tool registrar_triagem(dados) -> RAPIA grava voice_calls.triage (JSON) + vincula ao contato
tool verificar_disponibilidade(fila) -> há atendente? -> tool enfileirar(fila, prioridade)
RAPIA -> Pusher: "voice.queue.updated" -> Painel 360 mostra a chamada + triagem
IA mantém o paciente em espera (mensagem + áudio de espera publicado na sala; posição na fila opcional)
Atendente clica "Assumir" -> RAPIA faz lock atômico da chamada -> token JWT da sala
    -> RAPIA sinaliza o worker (RPC/room metadata) -> IA se despede e sai da sala
    -> atendente entra via WebRTC e conversa com o paciente (a ligação nunca caiu)
    
```

Cenário 3: Sem atendente, continuar pelo WhatsApp

```

verificar_disponibilidade = nenhum atendente (ou espera > limite configurado)
IA oferece o WhatsApp -> confirma o número (o caller ID pode ser fixo/oculto -> pede um celular)
tool enviar_whatsapp(numero, template_id, variaveis)
    -> RAPIA dispara o template (categoria UTILITY) pelo WhatsTemplateController existente
    -> cria vínculo voice_call <-> contato/sessão pendente com a triagem anexada
    
```

Paciente responde o template -> janela de 24h aberta -> sessão entra na fila de chat existente
-> atendente do chat já recebe a triagem feita por voz (contexto omnichannel)

Cenário 4: Callback

Paciente recusa o WhatsApp -> IA avisa o retorno -> tool `agendar_callback(numero, fila, triagem)`
RAPIA: `voice_callbacks` (status=pendente, tentativas=0, janela de horário, prioridade, expira_em)
Evento "atendente ficou online/disponível" (MapaAgentes/Pusher) -> job de distribuição:
lock atômico no callback mais antigo elegível -> oferta ao atendente (modal, timeout ex.: 30 s)
aceitou -> sala criada -> atendente entra -> `CreateSIPParticipant(tronco de saída, numero)`
recusou/timeout -> oferece ao próximo atendente disponível
Resultado: atendeu | não atendeu | ocupado | caixa postal -> reagenda (máx. N tentativas) ou encerra

6.3 Modelo de dados sugerido (migrations com a checagem de banco "rapia")

Tabela	Finalidade
voice_trunks	Troncos SIP por tenant (entrada/saída, IPs, credenciais cifradas, codec, id no LiveKit)
voice_numbers	DIDs → tenant, agente de IA padrão, fila padrão, horário de atendimento
voice_agents / voice_agent_versions	Agentes de IA e versões publicadas (prompt, modelo, voz, idioma, parâmetros, regras de transbordo)
voice_agent_tools	Functions: nome, descrição, JSON Schema, tipo (interna RAPIA HTTP externa), endpoint, auth, timeout, mensagem de espera
voice_triage_forms	Campos da triagem por agente/fila (JSON Schema) e regras de prioridade
voice_calls	Chamada: direção, origem (pstn/whatsapp/web), números, sala, status, agente IA, atendente, triagem (JSON), resultado, custos, tempos
voice_call_events	Linha do tempo: tocou, IA atendeu, tool X, enfileirou, assumida, transferida, encerrada (auditoria)
voice_queue_entries	Chamadas aguardando em fila de voz (prioridade, entrada, abandono)
voice_callbacks	Retornos pendentes, tentativas, janela, oferta/aceite, resultado
voice_recordings / voice_transcripts	Gravação (S3, via Egress), transcrição, resumo gerado, retenção/expurgo

7. Ambiente de desenvolvimento local (docker-compose + ngrok)

7.1 Resposta direta

O docker-compose local funciona. O ngrok não resolve o tronco da operadora. Uma chamada SIP tem duas partes: a **sinalização** (SIP, UDP/TCP 5060) e a **mídia** (RTP, portas UDP dinâmicas, ex.: 10000–20000). O ngrok só tunela HTTP/HTTPS e TCP (os endpoints TCP exigem cartão cadastrado, mesmo no plano free) e **não suporta UDP**. Na melhor hipótese, o INVITE chega via TCP e a chamada "completa", mas sem áudio. Além disso, o SDP anuncia o IP/porta de mídia do servidor, e atrás de NAT/CGNAT residencial isso não é roteável pela operadora. O WebRTC do navegador também depende de UDP (ou TURN).

Outros pontos do ambiente Windows: o Docker Desktop não oferece *network_mode: host* equivalente ao Linux, e publicar milhares de portas UDP por mapeamento é lento e instável. Rode os containers de mídia no **WSL2** (Ubuntu) ou em uma VM Linux.

7.2 Três formas viáveis de desenvolver

Opção	Como montar	Quando usar
A. 100% local, sem operadora	docker-compose (WSL2): livekit-server + redis + livekit-sip. Um softphone (Zoiper/MicroSIP/Linphone) na rede local disca para o LiveKit SIP. Worker e Laravel locais.	Dia a dia: prompts, tools, fila, Painel 360, handoff, callback (com tronco de saída apontando para o softphone).
B. EC2 de desenvolvimento (recomendada para o tronco real)	EC2 pequena (ex.: t3.medium) com IP elástico rodando livekit-server + livekit-sip + redis (compose, rede host). A operadora aponta o tronco de teste para esse IP. Worker e Laravel continuam na sua máquina: o worker conecta de dentro para fora e os webhooks do LiveKit chegam ao Laravel local via ngrok HTTP (uso correto do ngrok).	Testes ponta a ponta com a operadora, custo de poucos dólares por mês (ligue/desligue a instância).
C. LiveKit Cloud (plano gratuito/Build)	Projeto no LiveKit Cloud com SIP gerenciado. A operadora aponta para o endpoint SIP do projeto. Worker e Laravel locais.	PoC mais rápida possível e testes da fase 2 (WhatsApp Connectors). Atenção à residência dos dados.

7.3 Esqueleto de docker-compose (opção A/B)

```
# docker-compose.yml (rodar em Linux/WSL2 - network_mode host)
services:
  redis:
    image: redis:7-alpine
    network_mode: host
  livekit:
    image: livekit/livekit-server:latest
    command: --config /etc/livekit.yaml
    network_mode: host
    volumes: ["/livekit.yaml:/etc/livekit.yaml"]
    depends_on: [redis]
  sip:
    image: livekit/sip:latest
    network_mode: host
    environment:
      SIP_CONFIG_BODY: |
        api_key: devkey
        api_secret: <segredo-com-32+-caracteres>
        ws_url: ws://localhost:7880
        redis: { address: localhost:6379 }
        sip_port: 5060
        rtp_port: 10000-20000
        use_external_ip: true      # na EC2; false no local
    depends_on: [redis, livekit]

# livekit.yaml
port: 7880
redis: { address: localhost:6379 }
keys: { devkey: <segredo-com-32+-caracteres> }
rtc: { tcp_port: 7881, port_range_start: 50000, port_range_end: 60000, use_external_ip: false }
webhook: { api_key: devkey, urls: [ "https://<seu-tunel>.ngrok.app/api/voice/livekit/webhook" ] }
```

Os nomes de imagem, chaves e opções devem ser conferidos na documentação vigente do LiveKit no momento da montagem. O agente em Python roda com **python agent.py dev** apontando **LIVEKIT_URL/API_KEY/API_SECRET** para o servidor (local, EC2 ou Cloud).

8. Configuração dos agentes de IA dentro do RAPIA

É possível e é o desenho recomendado. O RAPIA passa a ser a fonte única de verdade da configuração, e o worker é um executor genérico e sem estado. Quando uma chamada chega, o dispatch do LiveKit envia ao worker os metadados (tenant, agent_id, versão) e o worker busca no RAPIA a configuração de runtime, montando dinamicamente o prompt e as tools.

8.1 Telas sugeridas

Tela	Configurações
Agentes de voz	Nome, objetivo, idioma, provedor/modelo (Gemini Live, OpenAI Realtime, pipeline), voz, temperatura, tempo máximo de chamada, silêncio máximo, saudação, aviso LGPD, frases de encerramento, sensibilidade de interrupção.
Prompt (versionado)	Editor com variáveis ({{nome_clinica}}, {{horario_atendimento}}, {{data_hoje}}), histórico de versões, rascunho x publicado, rollback, comparação entre versões.
Ferramentas (functions)	Catálogo de tools internas prontas (buscar/cadastrar paciente, horários, agendar, cancelar, enfileirar, WhatsApp, callback, transferir, encerrar) + tools HTTP personalizadas: nome, descrição para o modelo, JSON Schema dos parâmetros, método/URL, headers/credenciais (cofre cifrado), mapeamento de resposta, timeout, mensagem falada de espera, se exige confirmação do paciente.
Integrações de gestão	Conectores por sistema da clínica (adapter no Laravel): credenciais, mapeamento de especialidades/convênios/profissionais, regras de agenda. As tools chamam o RAPIA, que chama o ERP.
Triagem e transbordo	Formulário de triagem (campos obrigatórios), regras de prioridade, palavras de emergência, fila de destino, limite de espera, política WhatsApp/callback, horários.
Telefonia	Troncos SIP, números (DIDs), roteamento DID → agente/fila, tronco de saída para callback, gravação sim/não, música de espera. Ao salvar, o RAPIA sincroniza trunks e dispatch rules via API do LiveKit.
Teste (playground)	Botão "falar com o agente" pelo navegador (WebRTC, sem telefone) usando a versão em rascunho, com transcrição e log de tools ao vivo.

8.2 Cuidados de segurança e governança

- O modelo só chama tools da **lista branca** do agente. Credenciais ficam no RAPIA, nunca no prompt nem no worker.
- Tools de escrita (cadastrar, agendar, cancelar) exigem confirmação explícita do paciente e chave de idempotência.
- Toda execução de tool é registrada em `voice_call_events` (entrada, saída, latência, erro).
- Proteção contra *prompt injection* por voz: o prompt define limites de escopo, e a tool valida no servidor (ex.: só agenda para o CPF verificado nesta chamada).
- Publicação de uma versão do agente só por perfil autorizado (integração com SysPermissoes).

9. Produção na AWS: mesmo servidor ou instância separada?

9.1 Resposta direta

O LiveKit não precisa, e não deve, rodar no mesmo servidor do RAPIA. A comunicação entre eles é toda por HTTP (API/webhooks) e WebSocket, então ficam em instâncias separadas sem problema. A separação é recomendada por: perfil de carga diferente (mídia em tempo real é sensível a CPU e rede; um pico no PHP-FPM não pode picotar o áudio); exigência de IP público e faixas UDP amplas; escala independente; e isolamento de segurança (portas SIP expostas atraem varreduras e tentativas de fraude).

9.2 Topologia recomendada

Componente	Onde rodar	Observações
LiveKit Server + LiveKit SIP (+ Egress)	EC2 dedicada(s), sa-east-1, IP elástico, Docker com rede host (compose/systemd)	Família compute-optimized (c6i/c7i ou c7g). Dimensionar pelo número de chamadas simultâneas na PoC. Multi-nó compartilha Redis.

Componente	Onde rodar	Observações
SBC (se necessário)	Mesma VPC, EC2 pequena ou junto ao LiveKit SIP	Registro na operadora, normalização, failover de operadoras.
Voice Agent Workers	Swarm (serviço replicado) ou EC2/ECS	Só conexões de saída. Escala horizontal. Precisam de CPU (VAD, detecção de turno) e boa rede até o provedor de IA.
RAPIA (Laravel, Vue)	Como hoje (EC2 → Swarm/Portainer)	Novas rotas /voice/*, webhooks, jobs.
Redis do LiveKit	ElastiCache ou Redis dedicado	Preferível separado do Redis de filas do RAPIA.
Gravações	S3 (Egress)	Criptografia, bucket por ambiente, lifecycle de retenção (LGPD).

9.3 Por que o LiveKit não deve entrar no Swarm (e os workers sim)

O *routing mesh* do Swarm (ingress) faz NAT das portas publicadas. Isso quebra o anúncio de IP/porta do WebRTC e do SDP, e publicar dezenas de milhares de portas UDP no Swarm é impraticável. Dá para usar *mode: host* com restrição de nó, mas na prática vira uma instância dedicada com overhead extra. O recomendado é manter os nós de mídia fora do Swarm (Docker Compose/systemd em EC2 dedicada, gerenciável pelo Portainer como *standalone environment*) e colocar no Swarm apenas os workers de IA, que são stateless e só fazem conexões de saída.

9.4 Portas e segurança (Security Group)

Porta	Protocolo	Uso	Origem liberada
443	TCP	API/WebSocket do LiveKit (atrás de TLS: LB ou Caddy/Nginx) e TURN/TLS	Internet (atendentes)
7881	TCP	WebRTC sobre TCP (fallback)	Internet
50000–60000	UDP	Mídia WebRTC (navegadores, workers)	Internet
3478	UDP	TURN/UDP (se habilitado)	Internet
5060 (e/ou 5061 TLS)	UDP/TCP	Sinalização SIP	Somente IPs da operadora (e do SBC)
10000–20000	UDP	Mídia RTP do SIP	IPs de mídia da operadora

Adicione também: limite de chamadas simultâneas por tronco, bloqueio de destinos internacionais e de custo elevado no tronco de saída (fraude de *toll fraud*), alertas de custo e o domínio próprio (ex.: *voz.rapia...*) com certificado TLS.

9.5 Self-hosted x LiveKit Cloud

O LiveKit Cloud elimina a operação da camada de mídia (escala, TURN global, atualização) e já traz os Connectors do WhatsApp, mas cobra por minuto/participante e processa mídia fora da infraestrutura própria. O self-hosted dá controle total e custo previsível, com a operação ficando com a equipe. Uma estratégia prática: **PoC no Cloud, produção self-hosted na AWS**, reavaliando o modo híbrido quando chegar a fase WhatsApp. O código é o mesmo nos dois casos, só muda URL e credenciais.

10. Regras de negócio e pontos de atenção

- **Caller ID fixo ou oculto:** no cenário 3, sempre confirme o celular para o WhatsApp. Número fixo não tem WhatsApp.
- **Custo e categoria do template:** a mensagem pós-ligação é business-initiated (cobrada pela Meta). Aprove um template **utility** específico ("Olá {{1}}, conforme nossa ligação...") para evitar a categoria

marketing.

- **"Primeiro atendente online":** defina se vale só ficar online ou também ficar livre (sem chamada/chat em curso), se respeita as filas do atendente e o que acontece se ninguém aceitar (fila, escalonamento, SLA).
- **Janela de callback:** não retornar fora do horário de atendimento. Definir tentativas máximas, intervalo e expiração, e avisar o paciente por WhatsApp/SMS se não for possível retornar.
- **Abandono:** paciente desliga na fila → registrar e opcionalmente gerar callback automático (configurável).
- **Espera longa:** a IA pode reavaliar a oferta de WhatsApp/callback a cada X minutos de fila.
- **Emergência:** regras fixas no prompt e na triagem para sintomas graves, orientando a ligar 192 e registrando com prioridade máxima.
- **Gravação e LGPD:** aviso no início da chamada, finalidade, retenção por tenant, acesso auditado, DPA com o provedor de IA (Google/OpenAI) cobrindo dados de saúde e transferência internacional.
- **Regulação de telefonia:** retornos solicitados pelo próprio paciente não são telemarketing. Mesmo assim, apresente o número oficial da central e alinhe com a operadora o programa de chamadas verificadas da Anatel, se aplicável.
- **Indicadores:** TME, TMA, taxa de resolução pela IA, transbordos, abandono, callbacks efetivados, custo por chamada (telefonia + IA), integrados ao DashController/relatórios.

11. Riscos e mitigação

Risco	Impacto	Mitigação
Operadora só oferece tronco com REGISTER	Alto	Verificar antes; SBC Asterisk/Kamailio na frente do LiveKit SIP.
IA erra CPF, datas ou nomes pelo telefone (8 kHz, ruído)	Alto	DTMF para o CPF, confirmação explícita, validação de dígito, comparação de modelos na PoC.
Latência alta das tools (ERP lento)	Médio	Timeouts, mensagens de espera, cache de agenda/especialidades, tools assíncronas.
Build do Vue 2/Webpack 3 incompatível com o SDK	Médio	Bundle UMD como script externo; componente isolado.
Custo por minuto da IA acima do esperado	Médio	Medir na PoC; Gemini como padrão; limites de duração; pipeline alternativo.
Voz do WhatsApp dependente do LiveKit Cloud	Médio (fase 2)	Origem como atributo; modo híbrido; acompanhar o suporte self-hosted/SBC.
Dados de saúde com provedores de IA	Alto	DPA, minimização (a IA não precisa do prontuário), retenção, revisão jurídica.
Fraude no tronco de saída	Alto	Restrição de destinos, limites, SG por IP da operadora, alertas.
Nova stack (Python) para a equipe	Médio	Worker genérico e pequeno; regras no Laravel; documentação e imagem Docker padronizada.

12. Roadmap sugerido

Fase	Escopo	Entregáveis / critério de saída
0. PoC (2–3 sem.)	LiveKit Cloud ou EC2 dev; tronco de teste da operadora; worker Python com 2–3 tools mock; Gemini x OpenAI; livekit-client no sysweb.	Chamada real atendida pela IA com CPF por fala/DTMF; handoff para navegador; custo/minuto medido; decisão IP x SBC.

Fase	Escopo	Entregáveis / critério de saída
1. Fundação	Migrations; cadastros (trancos, números, agentes, tools, versões); API runtime para o worker; webhooks; infra de produção (EC2 LiveKit, workers).	Agente configurável pelo RAPIA atendendo em homologação.
2. Cenário 1	Adapters do sistema de gestão (paciente, agenda); tools de agendamento; gravação/transcrição; relatórios básicos.	Autoagendamento ponta a ponta em piloto com uma clínica.
3. Cenário 2	Filas de voz, Paine 360, handoff IA → humano, supervisão, espera.	Atendentes assumindo chamadas triadas pelo navegador.
4. Cenários 3 e 4	Integração WhatsApp (template + vínculo de sessão), fila de callback, distribuição atômica, originação SIP.	Fluxo completo sem perda de contato.
5. WhatsApp Calling	Chamadas de voz do WhatsApp entrando nas mesmas salas/agentes/filas (Connectors ou SIP).	Mesmo atendimento para PSTN e WhatsApp.

As durações das fases 1 a 5 dependem do tamanho da equipe e da API do sistema de gestão da clínica. Estime-as ao final da PoC, com os riscos principais já eliminados.

13. Conclusão e próximos passos

A necessidade descrita é tecnicamente viável com o stack atual e se encaixa na arquitetura do RAPIA. O RAPIA continua como cérebro de negócio (configuração, filas, contatos, WhatsApp, relatórios), o LiveKit vira a camada de mídia e telefonia, e um worker Python enxuto executa os agentes de IA. O desenho também prepara o terreno para a fase de voz no WhatsApp sem retrabalho.

Perguntas para a operadora (antes da PoC)

- O tronco pode ser entregue por IP (sem REGISTER)? Quais IPs de sinalização e mídia vocês usam?
- Codecs suportados (G.711 A-law?), DTMF por RFC 2833/4733, transporte UDP/TCP/TLS, SRTP?
- O número de origem (A) chega sempre? É permitido apresentar o número da central nas chamadas de saída?
- Quantidade de canais simultâneos, tarifação e bloqueios de destino disponíveis.
- É possível um número/tronco de testes apontando para o ambiente de desenvolvimento?

Decisões internas necessárias

- Self-hosted desde a PoC ou PoC no LiveKit Cloud?
- Provedor de IA padrão e contrato/DPA (Google Cloud Vertex AI x Gemini API; OpenAI/Azure OpenAI).
- Qual sistema de gestão da clínica será o piloto e qual API ele oferece para pacientes e agenda?
- Política de gravação, retenção e acesso (jurídico/LGPD).
- Aceite de um serviço em Python no ecossistema e responsável por ele.

Referências consultadas

- LiveKit — Telephony/SIP: docs.livekit.io/telephony (SIP REGISTER não suportado; DTMF; transferências).
- LiveKit — "Introducing LiveKit Connectors" (01/09/2026): WhatsApp Business Calling via AcceptWhatsAppCall, disponível no LiveKit Cloud.
- livekit/sip — PR #842: suporte a Opus e DTLS-SRTP/ICE-lite para WhatsApp calling via SIP (em andamento).
- ngrok — documentação de endpoints TCP (sem suporte a UDP; TCP exige cartão no plano free).

- Código do projeto RAPIA (sysapi/sysweb) e escopo_projeto.txt (27/02/2026).